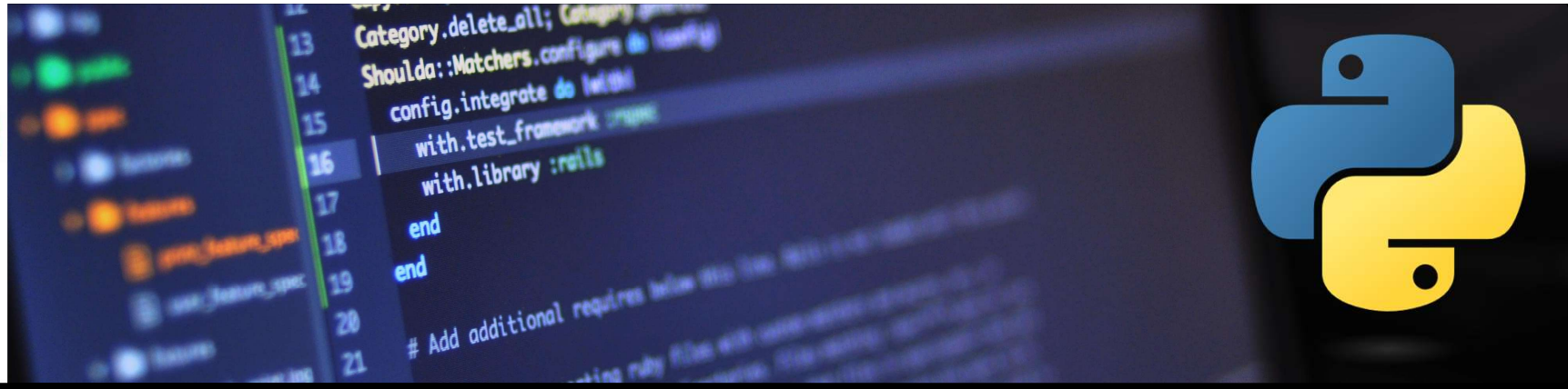
أكاديمية القاموس للأبحاث و الترجمة
QAMOUS ACADEMY FOR RESEARCH AND TRANSLATION



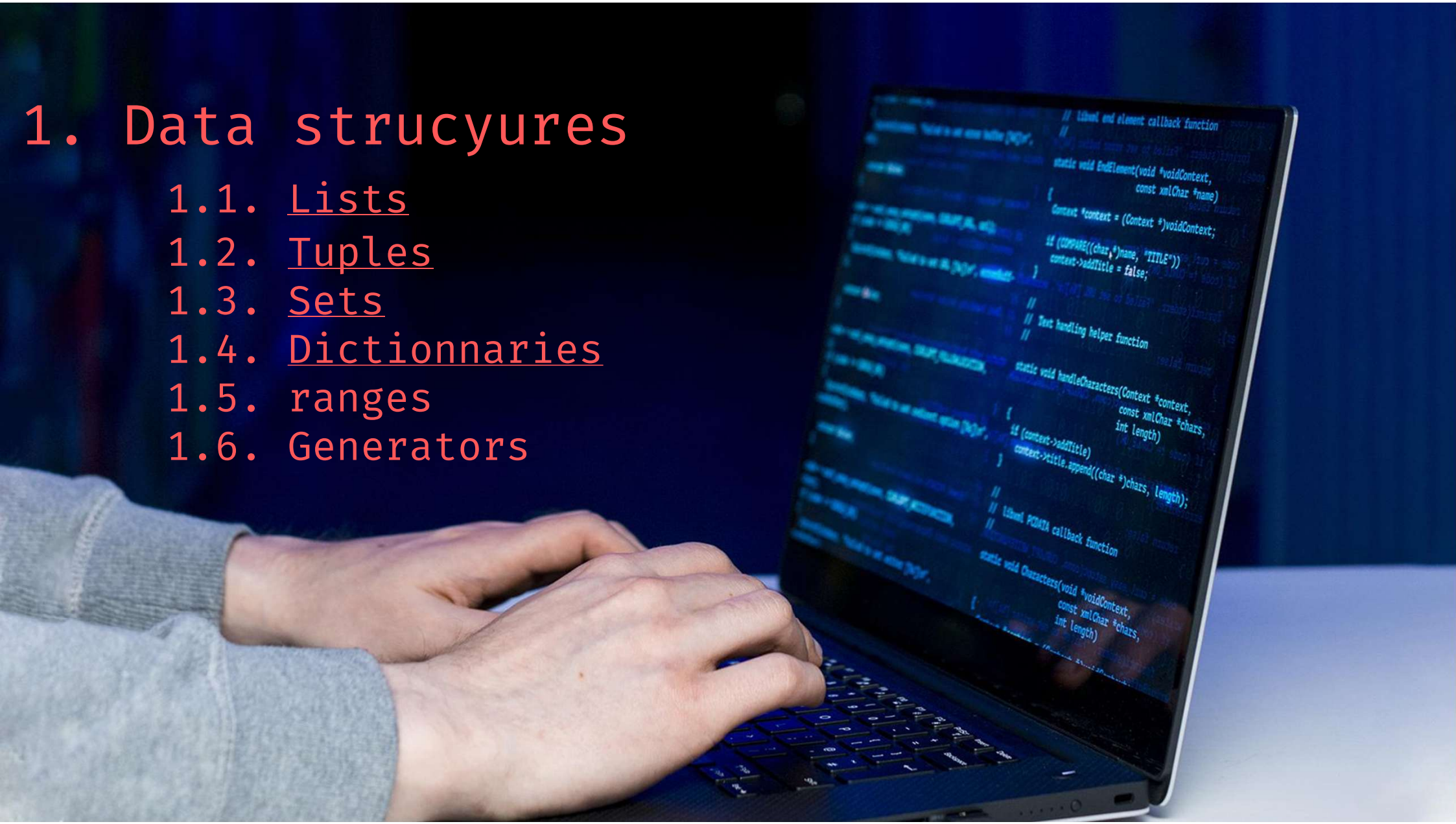
Python for beginners with applications

Write code from scratch in a clear & concise way, through a complete basic course.
From beginners to intermediate.

Lesson 03:

- *Data structures (lists, dictionary, tuple, set, range, generator)*
- *Functions & modules*
- *Mutable & immutable data types*
- *Exceptions and error handling*

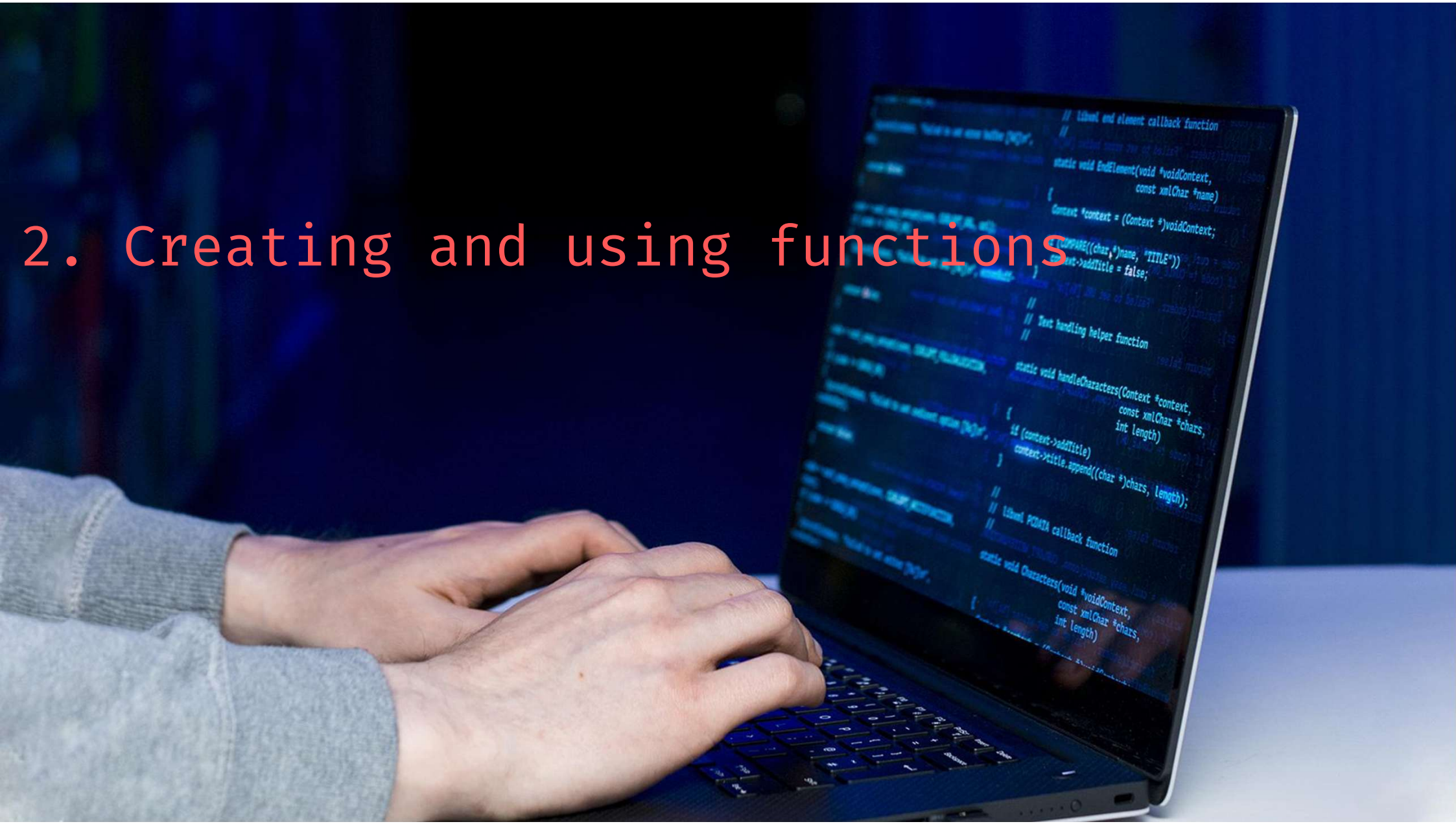
Abdesselam Filali 2025-12-03 19h00 Algiers time



1. Data strucyures

- 1.1. Lists
- 1.2. Tuples
- 1.3. Sets
- 1.4. Dictionnaries
- 1.5. ranges
- 1.6. Generators

2. Creating and using functions



Creating and using functions

Overview

A function is a block of code that performs a specific task when it is called, and functions are reusable, which means you can write a function once and use it as many times as you want.

Python has several built-in functions that you may be familiar with, including:

- print() prints an object to the terminal
- int() converts a string or number data type to an integer data type
- len() returns the length of an object

Function names include parentheses and may include parameter (arguments).

we'll go over how to define your own functions to use in your coding projects

Creating and using functions

Defining a function

A function is defined by using the def keyword, followed by a name of your choosing, followed by a set of parentheses which hold any parameters the function will take (they can be empty), and ending with a colon.

```
1    def hello():
2        print("Hello, World!")
```

Our function is now fully defined, but if we run the program at this point, nothing will happen since we didn't call the function.

```
1    def hello():
2        print("Hello, World!")
3    hello()
```

Level 1 : basic function without any parameter

1_01_funcLevel1.py > ...

```
1    def my_function():                # creating a silple function
2        print("Hello from a function")
3
4    my_function()                     # calling the function
5
```

Level 2 : function with parameters

1_01_funcLevel2Param.py > ...

```python
def greet1(name):    # This function greets to the person passed in as a parameter
    print(f"Hello, {name}")
greet1("Ahmed")

def greet2(name, age):    # This function has 2 parameters
    print(f"Hello, {name}! You are {age} years old.")
greet2("Amine", 25)
```

Level 3 : return one or more values

1_01_funcLevel3ReturnValues.py > ...

```
1    def calculate(a,b):
2        return a+b, a-b, a*b, a/b
3    print(calculate(10, 20))
```

Level 4 : default parameter values, parameter order

1_01_funcLevel4DefaultParam.py > ...

```python
def greet3(name, age=25):    # This function has 2 parameters, one by default
    print(f"Hello, {name}! You are {age} years old.")
greet3("Asmaa")
greet3("Asmaa", 27)

def greet4(name, age):    # This function has 2 parameters
    print(f"Hello, {name}! You are {age} years old.")
greet4(age=24, name="Reema")    # changing arguments order
```

Level 5 : docstring

1_01_funcLevel5Docstring.py > ...

```python
def divide(x, y):
    """
    Divide x by y and return the result.
    :param x: numerator
    :param y: denominator
    :return: division result
    """
    return x / y
result = divide(10, 2)
print(result)
```

Level 6 : variable scope

1_01_funcLevel6VarScope.py > ...

```
1  global_var = 10
2  def some_function():
3      local_var = 5
4      print(global_var + local_var)
5  some_function()
```

Level 7 : recursion

1_01_funcLevel7Recursion.py > ...

```python
def factorial(n):
    if n == 0:
        return 1
    else: return n * factorial(n - 1)

print(factorial(10))
```

Level 8 : Lambda

1_01_funcLevel8Lambda.py > ...

```python
VAT=0.19
VAT_Included = lambda x: x * (1 + VAT)
print(VAT_Included(100))
```

Level 9 : functional

1_01_funcLevel9Functional.py > ...

```python
def apply(func, x):
    return func(x)

def square(x):
    return x ** 2

result = apply(square, 5)
print(result)
```

Level 9 : Arbitrary positional parameters

1_01_funcLevel9ArbitraryPosition.py > ...

```python
def sum_all(*args):
    """ Sum all given arguments. """
    return sum(args)
total = sum_all(1, 2, 3, 4, 5)
print(total)
```

Level 10 : functional

1_01_funcLevel9Functional.py > ...

```python
def apply(func, x):
    return func(x)

def square(x):
    return x ** 2

result = apply(square, 5)
print(result)
```

Python Program to Count the Number of Vowels in a String

1_02_example.py > ...

```python
# Python Program to Count the Number of Vowels in a String
def count_vowels(input_string):
    vowels = "aeiouAEIOU"
    count = 0
    for char in input_string:
        if char in vowels:
            count += 1
    return count
user_input = input("Enter a string: ")
vowel_count = count_vowels(user_input)
print(f"The number of vowels in the string is: {vowel_count}")
```

Python Program to Find the GCD of Two Numbers

1_02_example2.py > find_gcd

```python
def find_gcd(a, b):
    while b:
        a, b = b, a % b
    return a
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
gcd = find_gcd(num1, num2)
print(f"The GCD of {num1} and {num2} is {gcd}.")
```

Functions – practice Example

Logical of a number

1_02_example3.py > ...

```
1   def logical(x):
2       if x == 1:
3           return 1
4       else:
5           return(x + logical(x-1))
6   result = logical(5)
7   print(result)
```

Creating and using functions

Functions – practice Example

Scope

1_02_example4.py > ...

```
1  def fun():
2      x = 100
3      print(x)
4  x=+1
5  fun()
```

Scope

1_02_example5.py > ...

```
1   x=10
2   def modify_x():
3       x=x+1
4       print(x)
5   modify_x()
```

1_02_example5.py > ...

```
1   x = 10
2   def modify_x(num):
3       return num + 1
4   x = modify_x(x)
5   print(x)
```

Scope

1_02_example5.py > ...

```
1   x=10
2   def modify_x():
3       x=x+1
4       print(x)
5   modify_x()
```

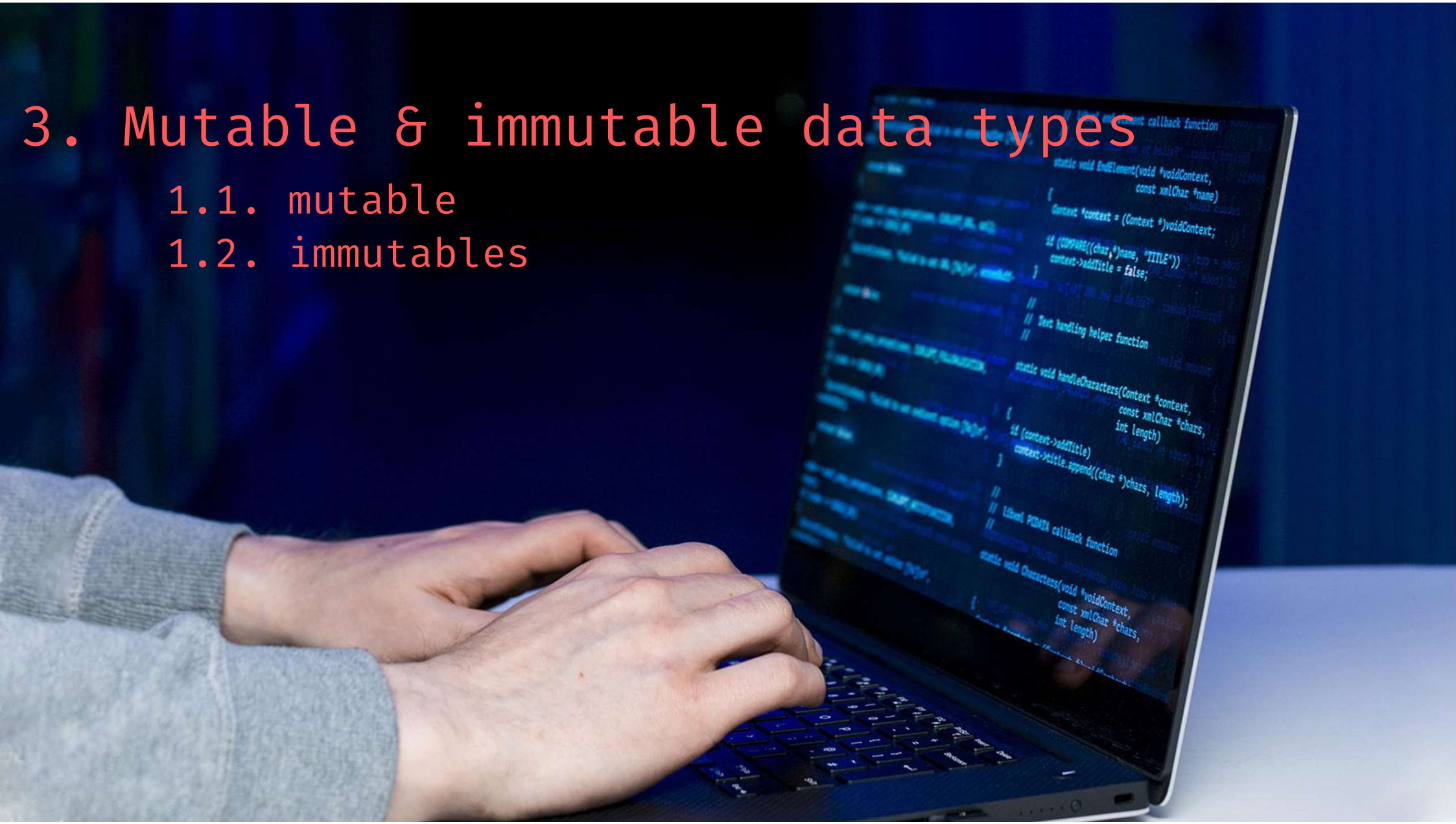
1_02_example5.py > ...

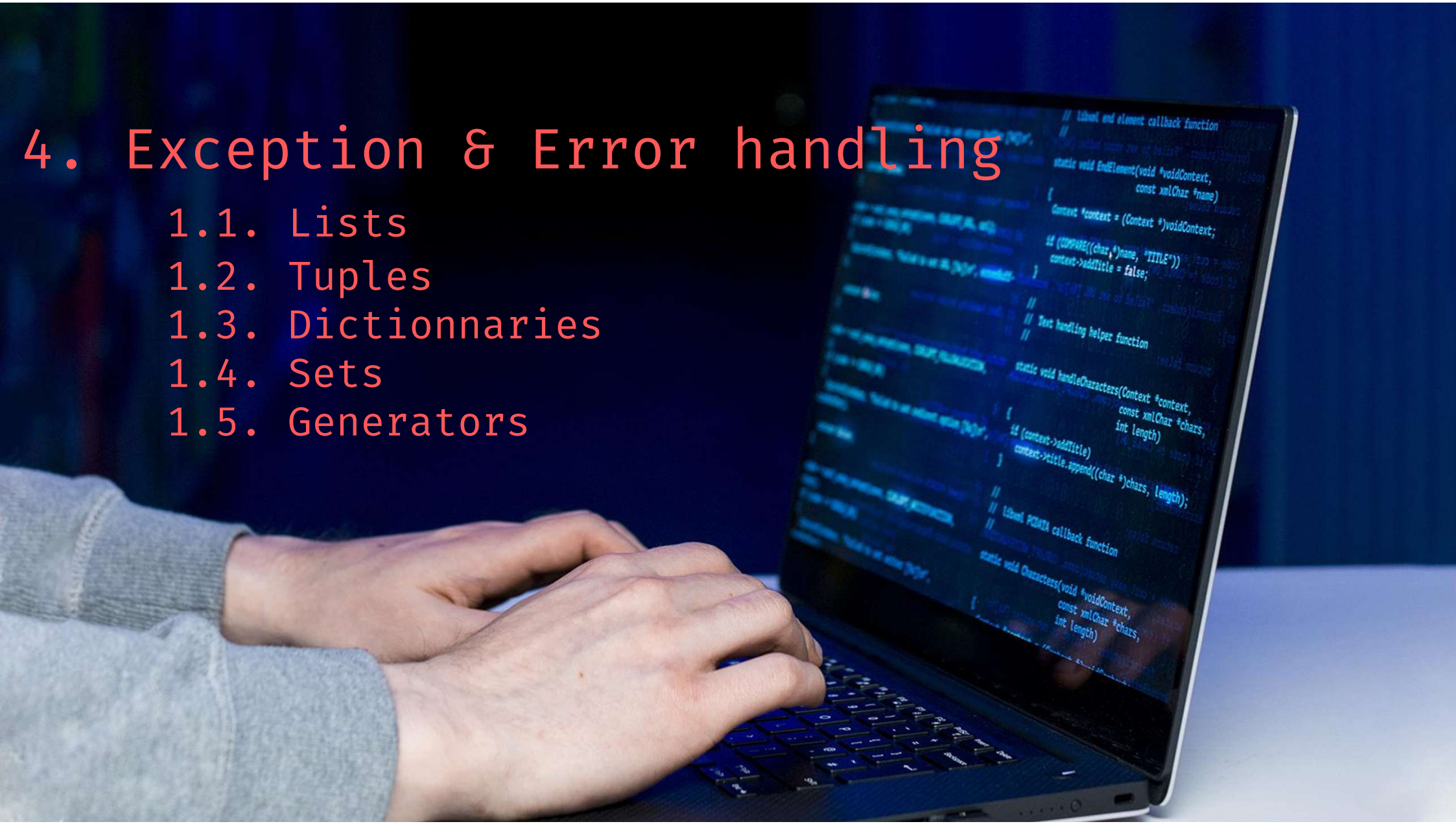
```
1   x = 10
2   def modify_x(num):
3       return num + 1
4   x = modify_x(x)
5   print(x)
```

3. Mutable & immutable data types

1.1. mutable

1.2. immutables





4. Exception & Error handling

1.1. Lists
1.2. Tuples
1.3. Dictionnaries
1.4. Sets
1.5. Generators

2.

File handling in Python

File handling in Python

Overview 1/2

File handling is an important part of any application.

Python has several functions for creating, reading, updating, and deleting files.

تعد معالجة الملفات جزءًا مهمًا من أي تطبيق.

لدى Python العديد من الدوال لإنشاء الملفات وقراءتها وتحديثها وحذفها.

https://www.w3schools.com/python/python_file_handling.asp

The key function for working with files in Python is the open() function.
The open() function takes two parameters; filename, and mode.
There are four different methods (modes) for opening a file:

"r" - Read - Default value. Opens a file for reading, error if the file does not exist
"a" - Append - Opens a file for appending, creates the file if it does not exist
"w" - Write - Opens a file for writing, creates the file if it does not exist
"x" - Create - Creates the specified file, returns an error if the file exists

In addition, you can specify if the file should be handled as binary or text mode
"t" - Text - Default value. Text mode
"b" - Binary - Binary mode (e.g. images)

File handling in Python

Open and read a file

Filename	Mode	Reading Method
"test.txt"	"rt"	read()

Code:

2_1_openFile_read.py > ...

```
1    myFile = open("test.txt", 'rt')
2    print(myFile.read())
```

Because "r" for read and "t" for text are the default values, you don’t need to specify them.

Practical Exercise: Read a file

Read a file:

- The file is in the same directory
- The file is in a different directory
- The file doesn’t exist
- Read and print it on screen

Complete path

Code:

2_04_openFile_read.py > ...

```
1   f = open("D://myfiles/welcome.txt", "r")
2   print(f.read())
```

Slash instead of back slash

Read part of a file

Filename	Mode	Reading Method
"test.txt"	"rt"	Read(n)

Code:

2_02_openFile_read.py > ...

```python
# Return the 6 first characters of the file
f = open("test.txt", "r")
print(f.read(6))
```

Practical Exercise: Read an image file

Read an image file

- Al-Aqsa.jpg (in the same directory)
- Binary file
- Read and print it on screen
- Show the image
- Remove the background (library)

Example

Read part of a file (lines)

Filename	Mode	Reading Method
"test.txt"	"rt"	Readline(n)

Code:

2_03_openFile_readline.py > ...

```
1   myFile = open("test.txt", 'rt')
2   print(myFile.readline())
3   print(myFile.readline())
4   print(myFile.readline())
5   for x in myFile:
6     print(x, end='')
7   myFile.close()
```

Create a file

To write to an existing file, you must add a parameter to the open() function:
"a" - Append - will append to the end of the file
"w" - Write - will overwrite any existing content

Code:

2_05_writeToFile.py > ...

```
f = open("test.txt", "a")
f.write("\nNow the file has more content!")
f.close()
#open and read the file after the appending:
f = open("test.txt", "r")
print(f.read())

f = open("file1.txt", "w")
f.write("Woops! I have deleted the content!")
f.close()

#open and read the file after the overwriting:
f = open("demofile3.txt", "r")
print(f.read())
```

File handling in Python

Create a file

To create a file, you must add a parameter to the open() function:
"x" - Create a new file
"w" - Create a new file id it doesn’t exist

Code:

2_06_createFile.py > ...

```
1   f = open("file1.txt", "x") # Create a new file
2   f = open("file2.txt", "w") # Create a new file if it doesn't exist
```

File handling in Python

Delete a file

To delete a file, you must use the os library

`os.remove(file.txt)` - Remove a file
`os.rmdir(folder1)` - Remove a directory, you can only delete an empty one

Code:

2_07_deleteFile.py > ...

```python
import os
os.remove("file2.txt")

if os.path.exists("file3.txt"):
    os.remove("file3.txt")
else:
    print("The file does not exist")
```

File handling in Python

Example

Writing to a file

Filename	Mode	Reading Method
"test.txt"	"rt"	read()

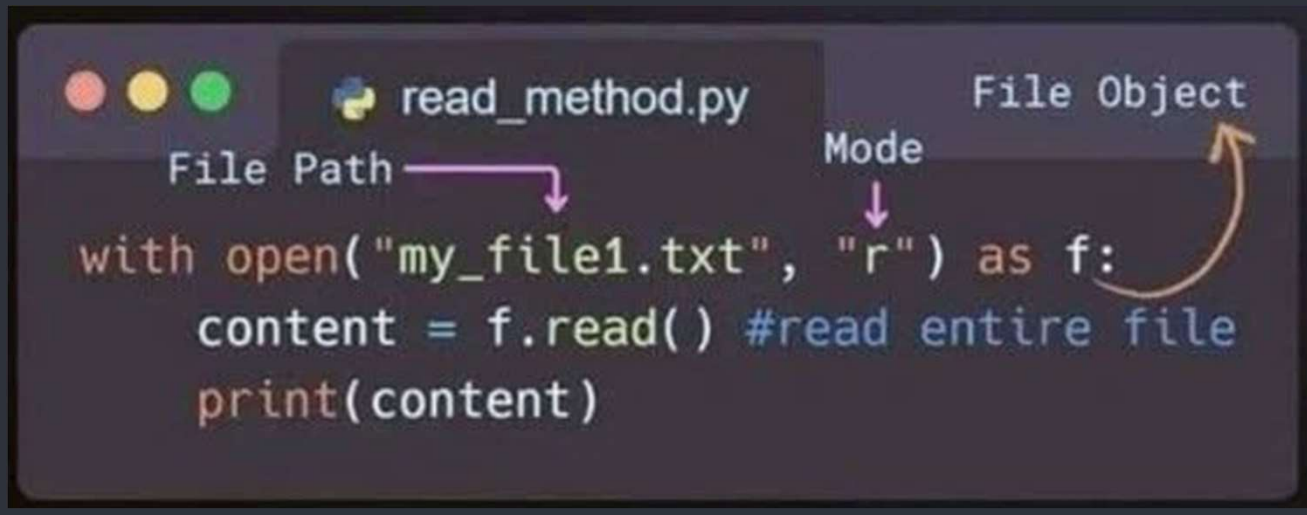
Code:

2_1_openFile_read.py > ...

```
1    myFile = open("test.txt", 'rt')
2    print(myFile.read())
```

File handling in Python

Tips reading a file



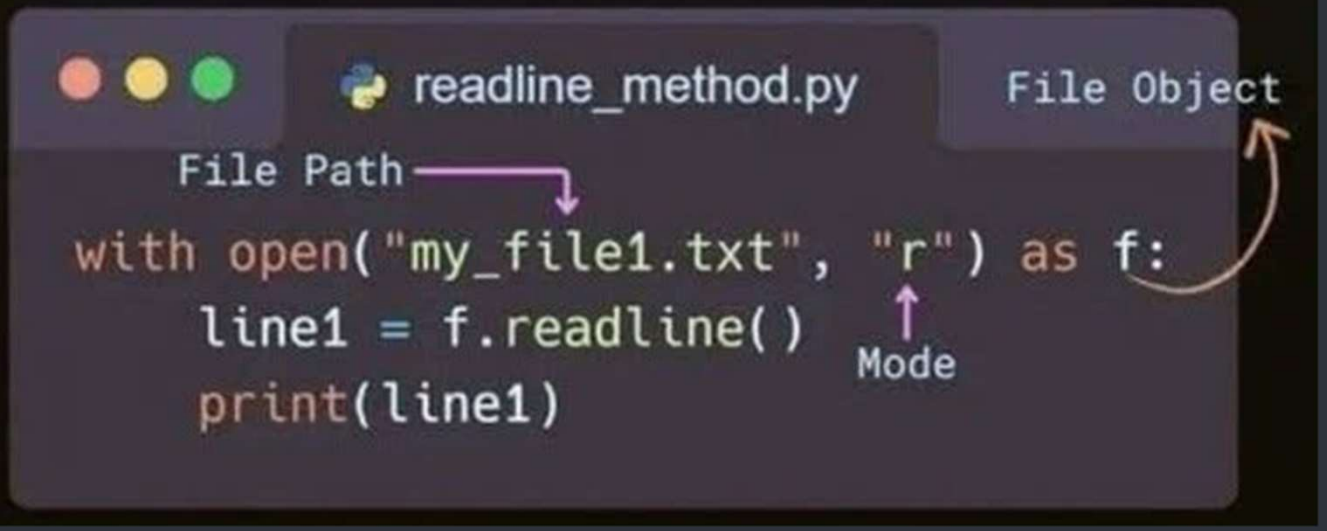
The read() method reads a specified number of characters (or bytes) from a file, or the entire file content if no size is specified.

Programming Language

File handling in Python

Tips reading a file

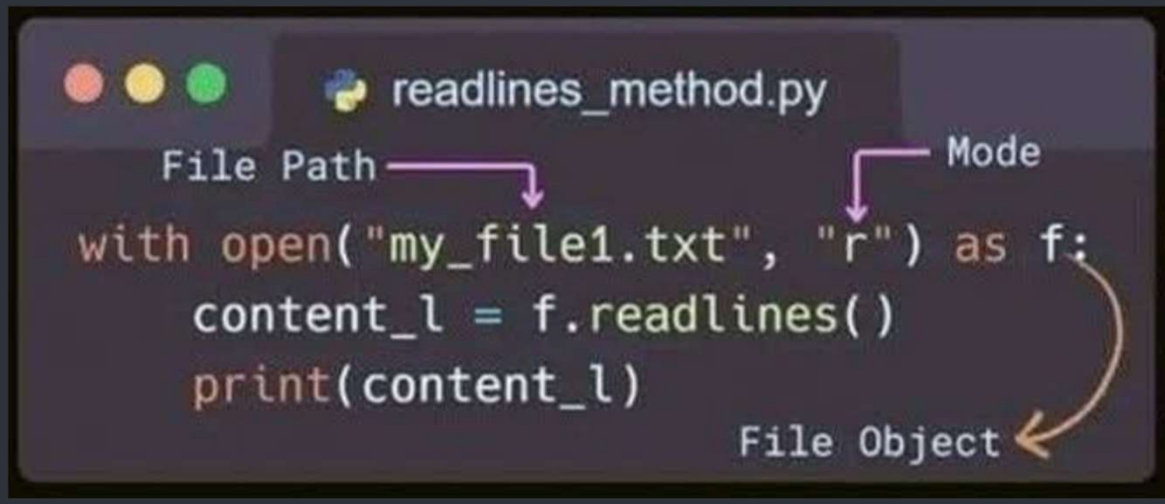
The readline() method in Python is employed to sequentially read one line at a time from a file.



Programming Language

File handling in Python

Tips reading a file

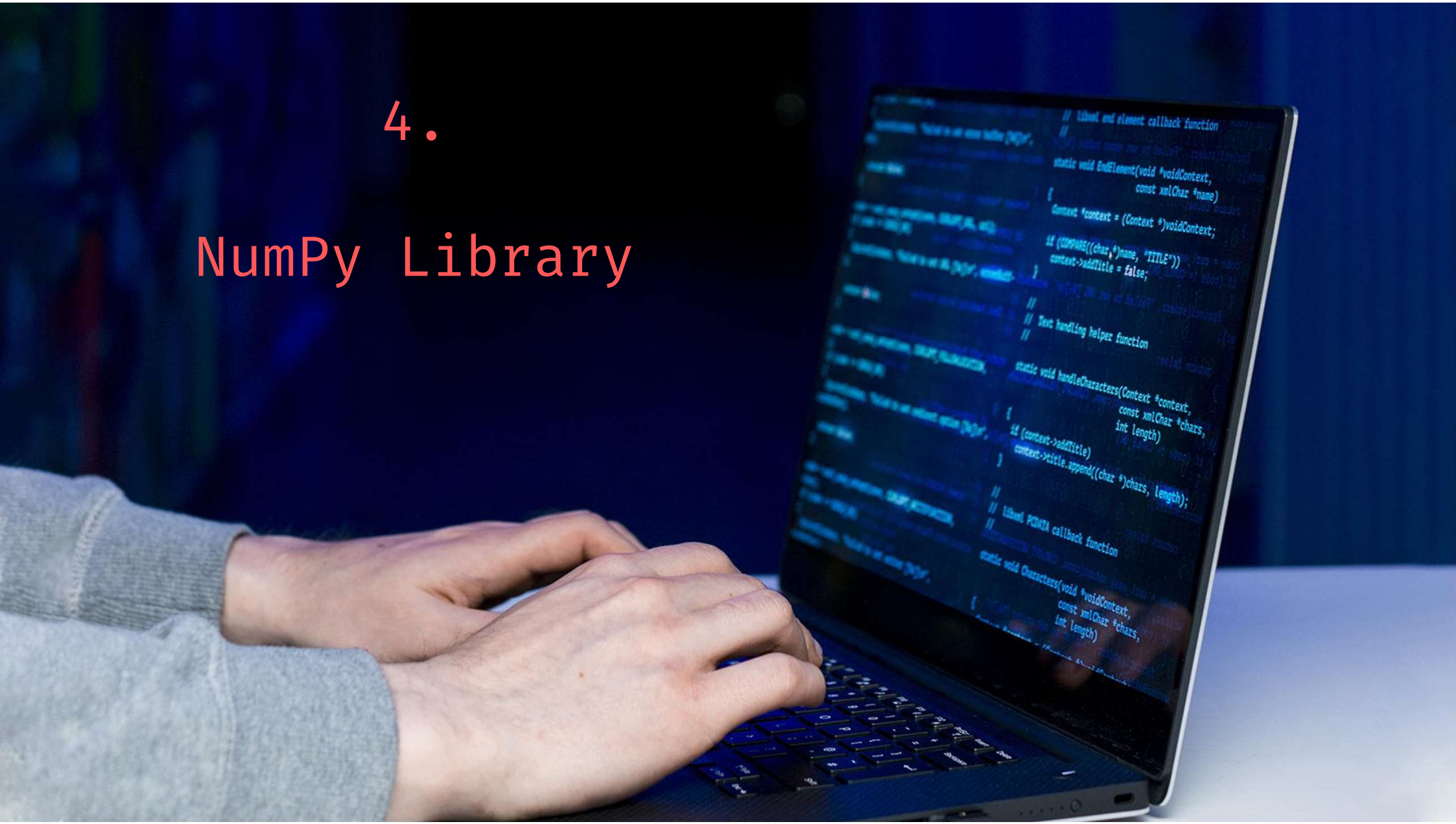


The readlines() method reads all lines from a file, returning them as a list where each element corresponds to a line in the file.

Programming Language

4.

NumPy Library




BREAKING NEWS
All Python Imports have 85% Tariffs from 15th April
Special extra 5% tariffs on Numpy & Pandas

What is NumPy?

NumPy (Numerical Python) is a fundamental powerful library for mathematical and numerical operations.
It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these arrays efficiently.

Why use NumPy?

- Much faster than Python lists for numerical tasks
- Used as the foundation for libraries like Pandas, TensorFlow, and Scikit-learn
- Makes matrix operations, statistics, and data transformations easy and efficient

installation

```
pip install numpy
```

What is NumPy Array?

A NumPy array is a powerful data structure that stores elements of the same data type in a grid-like format.
It's much faster and more efficient than Python lists for numerical operations..

Creating Arrays:

```
import numpy as np

np.array([1, 2, 3])          # From a list
np.zeros((2, 3))             # Array of zeros
np.ones((3, 3))              # Array of ones
np.arange(0, 10, 2)          # Range of numbers
```

Creating Array:

Code:

4_01_numpy.py > ...

```python
import numpy as np
arr = np.array([1, 2, 3, 4, 5])  # Creating a NumPy array
result = arr * 2  # Performing mathematical
                  # operations on the array
print(result)     #output [2 4 6 8 10]
```

Basic Array Operations:

Indexing and slicing:

```
arr = np.array([10, 20, 30, 40])
arr[1]          # 20
arr[1:3]        # [20 30]
```

Array Shape & Dimensions:

```
arr.shape       # Returns (rows, columns)
arr.ndim        # Returns number of dimensions
arr.size        # Total number of elements
```

Reshaping Arrays:

```
arr = np.array([[1, 2], [3, 4], [5, 6]])
arr.reshape(2, 3)   # Change the shape
```

Reshaping Arrays:

```
arr.dtype      # Get the data type of elements
```

Mathematical Operations:

Element-Wise operations:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b       # [5 7 9]
a * b       # [4 10 18]
a ** 2      # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])

np.sum(arr)       # 30
np.mean(arr)      # 10.0
np.max(arr)       # 15
np.std(arr)       # Standard Deviation
```

Reshaping Arrays:

Mathematical Operations:

Element-Wise operations:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b       # [5 7 9]
a * b       # [4 10 18]
a ** 2      # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])

np.sum(arr)       # 30
np.mean(arr)      # 10.0
np.max(arr)       # 15
np.std(arr)       # Standard Deviation
```

NumPy makes math easy — no need for loops! Just write it once and it works on the entire array.

Example using NumPy

Let's use NumPy in a real-world scenario: analyzing temperature data!
Example: Calculate Average Weekly Temperature

4_05_CalculateAvgWeeklyTemp.py > ...

```python
import numpy as np

# temperature in degrees celsius for 7 days
temps=np.array([29.3, 42.1, 18.8, 22.1, 17.6, 28.7, 33.4])

#average temperature
avg_temp=np.mean(temps)
print(f"Average temperature for 7 days: {avg_temp:.1f} degrees celsius")

#highest temperature
max_temp=np.max(temps)
print(f"Highest temperature for 7 days: {max_temp:.1f} degrees celsius")

#lowest temperature
min_temp=np.min(temps)
print(f"Lowest temperature for 7 days: {min_temp:.1f} degrees celsius")
```

NumPy

https://www.w3schools.com/python/numpy/default.asp

NumPy

#1 **np.array():** Create a NumPy array from a Python list or tuple.

#2 **np.zeros():** Create an array filled with zeros of a specified shape.

#3 **np.ones():** Create an array filled with ones of a specified shape.

#4 **np.arange():** Create an array with values within a specified range.

#5 **np.linspace():** Create an array with evenly spaced values over a specified interval.

NumPy

#6 **np.reshape():** Reshape an array into a new shape.

#7 **np.random.rand():** The rand() function is used to generate an array with random values between 0 to 1.

#8 **np.random.randn():** Generate an array of random numbers from a standard normal distribution. (close to zero)

#9 **np.random.randint():** Generate an array of random integers within a specified range.

#10 **np.mean():** Calculate the mean of array elements.

#11 **np.median():** Calculate the median of array elements.

NumPy

#12 `np.std()`: Calculate the standard deviation of array elements.

#13 `np.sum()`: Compute the sum of array elements.

#14 `np.min()`, `np.max()`: Find the minimum and maximum values in an array.

#15 `np.argmin()`, `np.argmax()`: Find the indices of the minimum and maximum values in an array.

#16 `np.dot()`: Compute the dot product of two arrays.

#17 `np.transpose()`: Calculate transpose of the array.

#18 `np.concatenate()`: Concatenate arrays along a specified axis.

NumPy

#19 **np.split()**: Split an array into multiple sub-arrays.

#20 **np.vstack()**, **np.hstack()**: Stack arrays vertically and horizontally.

#21 **np.unique()**: Find the unique elements in an array.

#22 **np.save()**, **np.load()**: Save and load arrays to/from disk.

#23 **np.clip()**: Clip (limit) the values in an array.

#24 **np.where()**: Return elements chosen from two arrays based on a condition.

#25 **np.linalg.inv()**: Calculate inverse of the matrix.

NumPy

#26 **np.linalg.det():** Calculate determinant of the matrix.

#27 **np.linalg.solve():** Solve a system of linear equations.

#28 **np.percentile():** Compute the nth percentile of the data.

#29 **np.corrcoef():** Compute the correlation coefficient between two arrays.

#30 **np.deg2rad(), np.rad2deg():** Convert angles from degrees to radians and vice versa.

#31 **np.argsort():** Return the indices that would sort an array.

#32 **np.searchsorted():** Find indices where elements should be inserted to maintain order.

NumPy

#33 **np.arcsin(), np.arccos(), np.arctan():** Inverse trigonometric functions.

#34 **np.linalg.eig():** Eigenvalues and eigenvectors of a square matrix.

#35 **np.linalg.svd():** Singular value decomposition.

ZeroDivisionError is raised when a program attempts to perform a division operation where the denominator is zero.

Syntax: ZeroDivisionError: division by zero

Why :

Direct Division by Zero

Code:

3_01_exceptionHandling.py > ...

```python
numerator = 10
denominator = 0
result = numerator / denominator
print(result)
```

Why : **Variable Initialization Issue**

Code:

3_01_exceptionHandling.py > denominator

```
1   denominator = 0
2   result = 20 / denominator
```

Why : **Conditional Statements Issue**

Code:

3_01_exceptionHandling.py > ...

```
1   denominator = 0
2   if denominator != 0:
3       result = 25 / denominator
```

Approach/Reason to Solve Zerodivisionerror

Use if-else

Code:

3_01_exceptionHandlingSol.py > ...

```python
numerator = 10
denominator = 0

if denominator != 0:
    result = numerator / denominator
else:
    print("Error: Cannot divide by zero")
```

Using Conditional Expression

3_01_exceptionHandlingSol.py > ...

```python
numerator = 10
denominator = 0

result = numerator / denominator if denominator != 0 else "Error: Denominator cannot be zero"
print(result)
```

Catching the Exception (Using Try-Except Block)

3_01_exceptionHandlingSol.py > ...

```python
numerator = 10
denominator = 0
try:
    result = numerator / denominator
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
```

Input a valid number

3_01_exceptionHandlingSolExample.py > ...

```python
while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except ValueError:
        print("Oops!  That was no valid number.  Try again...")
print(x)
```

Code:

3_02_errorHandling.py > ...

```python
# bad practice
def read_file(file_path):
    file = open(file_path, 'r')
    content = file.read()
    file.close()
    return content
print(read_file('textError.txt'))

# good practice
def read_file(file_path):
    try:
        with open(file_path, 'r') as file:
            return file.read()
    except FileNotFoundError:
        return "File not found."
print(read_file('text_Error.txt'))
```